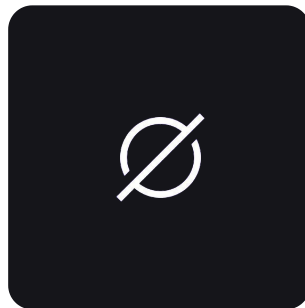


SECURITY REVIEW

FOR ENCIPHER



INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	M-01 Slippage Validation Failures	09
	M-02 Kafka Consumer Retry Mechanism	10
	I-01 Kafka missing SSL/TLS encryption and no auth	11
	I-02 Incomplete Balance Check in SolanaSolver	12
08	DISCLAIMER	13
09	ABOUT KANN AUDITS	14

AUDIT SUMMARY

Encifher engaged Kann Audits to perform a security review of the Encifher smart contracts. From 4 – 11 February 2026, a team of security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

4

ISSUES

CRITICAL	-
HIGH	-
MEDIUM	02
LOW	-
INFORMATIONAL	02

ENGAGEMENT DETAILS

PREPARED FOR	Encifher
PREPARED BY	Kann Audits
DATE REPORTED	12 February 2026
REVIEW PERIOD	4 – 11 February 2026

PROTOCOL OVERVIEW

Encipher is a Web3 privacy protocol focused on bringing confidential and compliant transactions to decentralized finance (DeFi) on the Solana blockchain. Instead of exposing sensitive details like transaction amounts or trading strategies publicly on-chain, Encipher uses advanced cryptography and off-chain secure computation to encrypt value while keeping key parts of DeFi composability intact.

03 - WHAT WE REVIEWED

SCOPE

AUDITED COMMIT	Private
FINAL COMMIT	Private

#	FILE
01	packages/consumer/*
02	packages/listener/*
03	packages/mixer/*
04	packages/pool-manager/*
05	packages/refund/*
06	packages/server/*
07	packages/solver/*

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL -	● HIGH -	● MEDIUM 02	● LOW -	● INFO 02
TOTAL 04 FIXED 04 ACKNOWLEDGED 00				

ID	TITLE	SEVERITY	STATUS
M-01	Slippage Validation Failures	● MEDIUM	● FIXED
M-02	Kafka Consumer Retry Mechanism	● MEDIUM	● FIXED
I-01	Kafka missing SSL/TLS encryption and no auth	● INFORMATIONAL	● FIXED
I-02	Incomplete Balance Check in SolanaSolver	● INFORMATIONAL	● FIXED

DETAILED FINDINGS

M-01

Slippage Validation Failures

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

Slippage values are calculated and displayed to users but never enforced during swap execution, leading to potential silent losses and misleading quotes.

■ CODE EXAMPLE

```
// Quote calculation
slippageLegOne: jupQuote.slippageBps,
slippageLegTwo: 50,

// Swap execution - missing slippageBps
orderUrl.searchParams.set("inputMint", inputMint);
orderUrl.searchParams.set("outputMint", outputMint);
orderUrl.searchParams.set("amount", amount);
```

■ IMPACT

- Users may receive significantly less than quoted amounts - Silent failures with no warning to users - Misleading quotes could harm user trust

■ RECOMMENDATION

- Pass 'slippageBps' to Jupiter API:

```
orderUrl.searchParams.set("slippageBps", slippageLegOne.toString());
```

- - Implement post-execution validation:

```
const actualOutput = await getTransactionOutput(signature);
if (BigInt(actualOutput) < BigInt(orderData.otherAmountThreshold)) {
  throw new Error('Slippage exceeded: expected , got ');
}
```

■ REMEDIATION

● FIXED

M-02

Kafka Consumer Retry Mechanism

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

Single shared Kafka consumer instance ('kafkaConsumer') across three consumers prevents proper error recovery, causing retries to fail and manual restarts to be required.

■ CODE EXAMPLE

```
// Shared consumer
export const kafkaConsumer = kafka.consumer({
  groupId: "consumer-group",
  sessionTimeout: 120 * 1000,
  heartbeatInterval: 10000,
});
// All three consumers call kafkaConsumer.run() independently
```

■ IMPACT

- Failed consumers cannot recover automatically - Retry mechanism fails repeatedly every 500ms - Reduced system availability and reliability

■ RECOMMENDATION

- Option 1 (recommended): Create separate consumer instances:

```
export const preDepositConsumer = kafka.consumer({ groupId: "predeposit-consumer-group" });
export const postDepositConsumer = kafka.consumer({ groupId: "postdeposit-consumer-group" });
export const solverConsumer = kafka.consumer({ groupId: "solver-consumer-group" });
```

- - Option 2: Use 'Promise.allSettled()' instead of 'Promise.all()' to handle failures without retrying entire group.

■ REMEDIATION

● FIXED

I-01

Kafka missing SSL/TLS encryption and no auth

SEVERITY

● INFORMATIONAL

STATUS

● FIXED

■ DESCRIPTION

All Kafka connections operate without SSL/TLS encryption and lack authentication mechanisms.

■ CODE EXAMPLE

```
// Kafka server and consumer configuration
// No SSL/TLS or SASL auth enabled
```

■ IMPACT

- All messages transmitted in plaintext - Wallet addresses, transaction IDs, and order details exposed to network eavesdropping

■ RECOMMENDATION

- Enable SSL/TLS encryption - Implement SASL authentication for all Kafka connections

■ REMEDIATION

● FIXED

I-02

Incomplete Balance Check in SolanaSolver

SEVERITY

● INFORMATIONAL

STATUS

● FIXED

■ DESCRIPTION

'isSolvable()' function performs an incomplete balance check and does not account for optional gas deposit transfers.

■ CODE EXAMPLE

```
async isSolvable(params: OrderParams): Promise<boolean> {
  const balance = await this.getConnection().getBalance(new PublicKey(currentPool)
    );
  const requiredAmount = BigInt(params.amount); // Missing gas
  return BigInt(balance) ≥ requiredAmount;
}
```

■ IMPACT

- Orders marked as solvable may fail due to insufficient funds - User transactions fail after acceptance - Wasted RPC calls and fees

■ RECOMMENDATION

Include gas deposit in balance validation:

```
let requiredAmount = BigInt(params.amount);
if (params.destinationGasDepositAddress) {
  requiredAmount += BigInt(DESTINATION_GAS_AMOUNT_LAMPORTS);
}
return availableBalance ≥ requiredAmount;
```

■ REMEDIATION

● FIXED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits